



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 8

Team No.:

Project Title: Linux Dynamic Memory Allocation and Memory Monitoring System

Team Members

Roll Number	Student Name	Signature
2520030347	Souradeepta Das	
2520030119	Shreyas K.	
2520030493	Nava Harsha	
2520030122	Akshith Reddy	

1. Abstract

This project develops a custom user-space dynamic memory allocator and a real-time Linux memory monitoring tool aligned with core systems programming concepts. Dynamic memory management governs process performance, yet standard libraries obscure kernel interactions, heap fragmentation, and virtual address mappings.

Our solution implements a user-space memory management library in C using low-level Linux system calls (brk, sbrk, and mmap) to allocate, split, and coalesce heap chunks via explicit free-lists. Concurrently, a monitoring subsystem parses Linux Virtual File System (VFS) nodes (/proc/meminfo, /proc/[pid]/statm, /proc/[pid]/maps) via unbuffered file I/O to inspect process address space layouts, page allocations, and Resident Set Size (RSS). Concurrency is managed using POSIX threads and mutex locks to guarantee thread-safe allocations, while POSIX signals deliver asynchronous threshold alerts when memory usage exceeds configured limits.

The outcome is a dual-utility suite: a high-efficiency heap allocation library benchmarked against native glibc malloc, coupled with a real-time terminal telemetry monitor visualizing process memory dynamics and kernel resource transitions.

2. Problem Statement

Standard dynamic memory allocators abstract kernel-level operations, preventing detailed visibility into process address space modifications, internal/external fragmentation, and page fault behaviors. Additionally, monitoring granular process heap expansions in real time requires low-level interactions with the Linux kernel and virtual filesystem. This project addresses the challenge of engineering a custom, thread-safe dynamic memory allocator from the ground up while building a live system monitoring tool using Linux system calls, VFS abstractions, IPC signals, and concurrency primitives.

3. Objectives

- 1. System Calls & Memory Management (CO-1, CO-4):** Implement custom user-space `malloc()`, `calloc()`, `realloc()`, and `free()` routines utilizing `brk()`, `sbrk()`, and `mmap()` system calls with chunk coalescing to reduce fragmentation.
- 2. Process Address Space & VFS Inspection (CO-4, CO-5):** Build a real-time memory telemetry monitor by parsing the `/proc` pseudo-filesystem via unbuffered Linux file I/O system calls.
- 3. Concurrency & Thread Safety (CO-6):** Integrate POSIX mutex locks and synchronization primitives to safeguard free-list structures from race conditions during concurrent multi-threaded requests.
- 4. Signals & Asynchronous IPC (CO-3):** Implement POSIX signal handlers (`SIGUSR1`, `SIGALRM`) to deliver real-time asynchronous notifications when process memory thresholds are breached.
- 5. Process Control & Profiling (CO-2, CO-4):** Evaluate and benchmark memory allocation throughput and resident memory footprint against glibc `malloc` across distinct process lifecycles.

4. Proposed Methodology

Memory Allocation Subsystem (CO-1, CO-4): Interacts across the user space and kernel space boundary using `sbrk()/brk()` for standard heap segments and `mmap()` for large chunk mappings. Free memory blocks are tracked using an explicit doubly linked list with boundary headers and footers to enable First-Fit/Best-Fit allocation and coalescing.

Concurrency & Synchronization Layer (CO-6): Multi-threaded allocation requests are serialized using `pthread_mutex_t` locks surrounding critical free-list traversals, preventing data corruption and deadlocks.

VFS Telemetry Subsystem (CO-5): A background monitoring daemon reads `/proc/[pid]/statm`, `/proc/[pid]/status`, and `/proc/meminfo` using unbuffered system calls (`open`, `read`, `close`) to extract `VSize`, `RSS`, and page table footprints.

Asynchronous Alerting & Process Management (CO-2, CO-3): Uses POSIX signal mechanisms (`sigaction`, `kill`) to notify parent and child monitoring processes when heap consumption reaches critical limits.

Benchmarking & Debugging (CO-4): The suite is evaluated using Valgrind, GDB, and timing harnesses to compare latency and fragmentation against standard library allocators.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
brk(), sbrk(), mmap(), munmap()	Requests and adjusts the heap segment break pointer and maps anonymous virtual memory pages directly from the kernel.
/proc VFS, open(), read(), close()	Extracts live memory telemetry, page mappings, and process address space metrics via unbuffered file descriptors.
pthread_create(), pthread_mutex_t	Manages concurrent memory allocation and protects shared free-list data structures from race conditions.
sigaction(), kill(), SIGUSR1, SIGALRM	Dispatches and handles asynchronous notifications for high-memory threshold alerts and periodic polling routines.
Process Lifecycle (fork(), waitpid())	Spawns and manages distinct test processes and background monitoring daemons across lifecycle states.
Address Space Layout & Paging	Analyzes virtual-to-physical address mappings, resident memory pages, internal/external fragmentation, and page faults.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030347	Souradeepa Das	Core allocator engine design; implementing sbrk()/mmap() system call interfaces, free-list traversal, and block coalescing/splitting logic.
2520030119	Shreyas K.	Linux VFS telemetry module; implementing low-level file I/O parsers for /proc/[pid]/statm and /proc/meminfo to capture RSS and address space metrics
2520030493	Nava Harsha	Concurrency and synchronization engine; implementing POSIX mutex locking, thread-safe memory wrappers, and multi-threaded stress testing.
2520030122	Akshith Reddy	Process lifecycle management and IPC; developing POSIX signal handlers (SIGUSR1/SIGALRM) for alerts, CLI formatting, and glibc benchmarking.

7. Tools / Platforms / Software Used

(List the programming languages, operating system, libraries, tools, and software planned for the project.)

Tool / Platform / Software	Purpose
Linux / Ubuntu	Target operating system environment and POSIX system call execution platform.
C / C++	Low-level systems programming languages for pointer arithmetic, data structure layout, and API development.
POSIX APIs (pthread, signal, unistd)	Multi-threading, asynchronous signal routing, and core OS service invocations
GCC & GDB	System software compilation, symbol debugging, and memory layout inspection.
Valgrind	Dynamic analysis tool for verifying leak detection, memory errors, and fragmentation profiling.

8. Expected Outcome

A functional, thread-safe user-space dynamic memory allocation library (.so) implementing custom malloc, calloc, realloc, and free operations with fragmentation control (CO-1, CO-4, CO-6).

A terminal-based real-time telemetry dashboard that parses Linux VFS /proc interfaces to display virtual memory size, RSS, and heap boundary shifts (CO-5).

An asynchronous event notification system utilizing POSIX signals for memory warning thresholds (CO-3).

Detailed benchmark reports measuring throughput, fragmentation ratios, and latency metrics compared to standard glibc allocation routines across varying process workloads (CO-2, CO-4).

9. GitHub Repository

GitHub Repository Link: https://github.com/soup-roop/SYSCORE_OSSP/tree/main/ForgeOS

Repository Created:

☐ Yes ☐ No

Faculty Name and Signature

Faculty Name: DR. MUNIRAJU VADLAMUDI

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____